FORM 2
THE PATENTS ACT, 1970
(39 of 1970)
&
The Patents Rules, 2003

COMPLETE SPECIFICATION
(See section 10 and rule 13)

---

1. TITLE OF THE INVENTION

SYSTEM AND METHOD FOR CROSS-RUNTIME DUAL-MODE POST-QUANTUM DATA ISOLATION AND CRYPTOGRAPHIC INGESTION

2. APPLICANT(S)

(a) **NAME:** QuantaLabs Private Limited
(b) **NATIONALITY:** India
(c) **ADDRESS:** [Insert Registered Company Address, City, State, PIN Code, India]

3. PREAMBLE TO THE DESCRIPTION

[For Complete Specification:]
"The following specification particularly describes the invention and the manner in which it is to be performed."

(Note: The Description should ideally start on a new page when printed/converted to PDF)

4. DESCRIPTION

Field of Invention
The present invention relates generally to cybersecurity and distributed cryptographic processing architecture. More specifically, the invention relates to a zero-trust, post-quantum cryptographic (PQC) infrastructure that enforces client-side cryptographic key lifecycle isolation across heterogeneous runtime environments (including WebAssembly and native runtimes) and enables verifiable network data ingestion and cryptographic receipt generation without exposure of private key material or unencrypted plaintext to network servers or intermediary gateways.

Background of the Invention
In modern computing architectures, enterprise data protection relies heavily on traditional public-key cryptographic algorithms, primarily Rivest-Shamir-Adleman (RSA) and Elliptic Curve Cryptography (ECC). These classical asymmetric algorithms depend on the computational intractability of mathematical puzzles, such as prime factorization and discrete logarithms. However, the advent of quantum computing breaks these underlying mathematical assumptions. Specifically, Shor's algorithm running on a sufficiently powerful quantum computer will render traditional RSA and ECC encryption systems vulnerable to polynomial-time decryption.

Presently, nation-state adversaries and sophisticated malicious threat actors are actively deploying "Harvest Now, Decrypt Later" (HNDL) attack methodologies. In an HNDL interception scheme, adversaries systematically exfiltrate and store encrypted enterprise network transmissions today, with the explicit intention of retroactively breaking the encryption once quantum computational resources attain sufficient cryptographic capability. Consequently, data possessing long-term confidentiality mandates—such as healthcare records, financial ledger entries, intellectual property, and government communications—is already at significant risk of future exposure.

While national algorithmic standards bodies (such as the National Institute of Standards and Technology - NIST) have standardized Post-Quantum Cryptography (PQC) mathematical functions such as Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM, formally Kyber), transitioning enterprise infrastructure to utilize these algorithms presents acute technical challenges in prior art implementations:

1. **Lack of Zero-Trust Architectural Isolation:** Many contemporary cloud cryptographic services rely on centralized encryption architectures (such as Hardware Security Modules or managed Key Management Services). In these prior art systems, either raw plaintext data is transmitted over networks to an external server for encryption, or private keys are generated and held within a third-party managed server environment. This violates strict zero-trust operational mandates by requiring trust in external network endpoints, making workloads vulnerable to gateway insider threats or data leaks at the network ingestion layer.
2. **Runtime Heterogeneity and Language Barrier Fragmentation:** Enterprise software systems operate across radically diverse execution runtimes—ranging from server-side Node.js and Python backend microservices to front-end browser environments running JavaScript. Existing PQC implementations are largely isolated to low-level C, C++, or pure Rust native binaries. Integrating these cryptographic routines into higher-level interpreted runtimes (like Node.js or browsers) typically necessitates insecure cross-language bindings, complex compilation pipelines, or duplicated, non-interoperable cryptographic logic that increases attack surfaces and memory vulnerability risks.
3. **Inflexible Key Lifecycle Semantics for Immutable Compliance:** Existing cryptographic libraries treat encryption identically regardless of operational use cases. For long-term audit logging (e.g., healthcare HIPAA audit records, compliance trails), data must be permanently sealed such that even an attacker compromising the original client machine cannot decrypt historical records. Prior art general-purpose libraries provide no architectural hardware or memory lifecycle enforcement to deterministically destroy key material immediately upon encryption to achieve non-repudiable permanent sealing.
4. **Lack of Cryptographic Ingestion Authenticity and Audit Receipts:** When encrypted payloads are ingested by enterprise API gateways, traditional application backends either accept raw unverified binary streams or attempt to decrypt the payloads for structural validation. Prior art API endpoints lack an isolated ingestion mechanism that can cryptographically validate structured post-quantum ciphertext format schemas, track billing and throughput telemetry, and generate immutable proof-of-ingestion receipts while operating wholly blind to the underlying encryption keys and plaintext.

Therefore, there exists a critical, unfulfilled need in the cybersecurity domain for a computer-implemented technical system that combines an optimized post-quantum cryptographic engine with structured dual-mode memory lifecycle semantics, seamlessly executable across disparate client runtimes via uniform compilation bridges, partnered with a network ingestion infrastructure that issues immutable cryptographic audit receipts without ever acquiring key exposure or plaintext readability.

Object of the Invention

- **Primary Object:** The main object of the present invention is to provide a zero-trust, post-quantum cryptographic processing system that ensures asymmetric private keys and unencrypted plaintext data never exit the local client execution memory or traverse network boundaries.
- **Second Object:** To provide a dual-mode cryptographic engine operable in a first ephemeral "Vault Mode" wherein ephemeral post-quantum asymmetric key material is deterministically generated, applied for encryption, and irrevocably released from memory within a single operational function scope without retention, and a second persistent "Secure Mode" wherein caller-held key material is managed for local end-to-end decapsulation and decryption.
- **Third Object:** To establish a cross-runtime compilation architecture that bridges a unified native cryptographic processing module into both WebAssembly (WASM) binaries for browser and Node.js JavaScript execution environments, and native foreign function interface (FFI) compiled extensions for Python execution runtimes, guaranteeing interoperable string ciphertext token structures across platforms.
- **Fourth Object:** To provide an isolated network API gateway ingestion framework configured to inspect and validate structured cryptographic ciphertext schemas via deterministic prefix tokens without possessing cryptographic decapsulation key material.
- **Fifth Object:** To provide a system for automatically generating and issuing non-repudiable, timestamped cryptographic receipts upon successful ciphertext payload ingestion to serve as verifiable compliance audit trails.

Summary of the Invention
The present invention relates to a computer-implemented system and method for cross-runtime post-quantum cryptographic data isolation and verifiable network ingestion. The system operates as an integrated zero-trust architecture comprising three synergistic layers: a core cryptographic execution engine, a heterogeneous runtime binding layer, and an untrusted network API gateway ingestion service.

In accordance with an embodiment of the invention, the **Core Cryptographic Engine** executes wholly within the local memory address space of a client machine. The engine implements a Key Encapsulation Mechanism and Data Encapsulation Mechanism (KEM/DEM) hybrid scheme. A lattice-based key encapsulation algorithm (ML-KEM / Kyber-1024) encapsulates an asymmetric shared secret, which sequentially parameterizes an authenticated symmetric cipher (Advanced Encryption Standard with Galois/Counter Mode, AES-256-GCM) to encapsulate the plaintext payload using a dynamically generated random nonce.

The core engine enforces two distinct operational memory lifecycles:

- In a **First Ephemeral Operational Mode ("Vault Mode")**, invoked for non-repudiable audit trails and immutable logs, the engine programmatically instantiates an ephemeral post-quantum keypair within a bounded function scope. Upon encapsulating the symmetric secret and generating the encrypted ciphertext, the engine immediately and deterministically executes a memory release and deallocation of the corresponding ephemeral private key prior to returning the ciphertext payload. This ensures the payload cannot be decrypted by any computational entity, including the client runtime itself.
- In a **Second Persistent Operational Mode ("Secure Mode")**, invoked for bidirectional end-to-end data exchange, the engine instantiates a persistent post-quantum keypair that is returned to the user runtime memory. Encrypted payloads are generated against targeted recipient public keys and can be locally decapsulated exclusively within the client memory of an authorized private key holder.

In another aspect of the invention, the **Runtime Binding Layer** overcomes language ecosystem fragmentation by compiling the single core cryptographic engine into two optimized structural targets:

- A **WebAssembly (WASM) Bridge** that injects compiled post-quantum assembly directly into JavaScript and TypeScript runtime bundles (browser and Node.js environments), incorporating runtime capability negotiation to detect cryptographic instructions without external C++ compilation dependencies.
- A **Native Foreign Function Interface (FFI) Bridge** that wraps the engine into native machine-code libraries executable inside Python host processes. Both runtime targets format output ciphertexts into an interoperable, version-tagged structured schema comprising cleartext prefix flags, Base64-encoded KEM ciphertext, a Base64-encoded initialization nonce, and Base64-encoded AES-GCM ciphertext.

In a further aspect of the invention, an **Isolated Network Ingestion Service (Gateway)** receives network requests containing client-generated cryptographic payloads. To enforce zero-trust integrity, the ingestion service contains zero cryptographic decapsulation routines and zero private key storage. The service performs structural lexical inspection on incoming payloads to confirm the presence and formatting of authorized scheme prefix tags (`QZ_VAULT_V1:`, `QZ_SECURE_V1:`), immediately rejecting any plaintext or non-conforming data transmission at the network boundary. Upon successful cryptographic schema validation and authenticated usage metering, the server generates an immutable cryptographic receipt token comprising an unforgeable receipt identifier, exact timestamp data, algorithm identifier tags, and ingested byte-count metadata, returning said receipt to the client as an verifiable, tamper-evident compliance artifact.

Brief Description of the Accompanying Drawings
The architectural execution and technical effects of the invention will be understood with reference to the following informal drawing schemas:

- **Figure 1** is a comprehensive architectural block diagram illustrating the multi-tiered cryptographic system spanning the core native engine, cross-runtime binding bridges, developer SDK abstractions, and the untrusted network ingestion gateway.
- **Figure 2** is an execution flow diagram illustrating the precise computational key lifecycle and deterministic memory deallocation routine executed during the Ephemeral Vault Operational Mode.
- **Figure 3** is an execution flow diagram illustrating the key encapsulation, network transmission, and local decapsulation workflow during the Persistent Secure Operational Mode.
- **Figure 4** is a signaling sequence diagram detailing the network protocol interactions between client runtime binding layers and the API gateway for zero-trust payload schema validation and cryptographic audit receipt generation.

Detailed Description of the Invention
The present invention discloses a developer-native, post-quantum encryption infrastructure specifically engineered to provide demonstrable technical effects in cybersecurity memory management, network boundary protection, and heterogeneous runtime interoperability.

Hardware Architecture and Technical Implementation
To achieve the aforementioned technical effects, the cryptographic engine executes on a physical computing apparatus comprising at least one hardware processor, a non-volatile memory storing the runtime binding layer instructions, and a volatile memory (RAM) where the ephemeral cryptographic registers are instantiated. The untrusted network API gateway server similarly comprises server-class hardware processors, network interface controllers (NIC) for receiving the structured ciphertext over physical network transmission mediums, and hardware-backed clock generators for issuing the immutable timestamped cryptographic audit receipts. By actively zeroizing specific physical RAM registers upon completion of the encapsulation routines, the system produces a tangible improvement in the security state of the underlying physical hardware, preventing data extraction via memory dumping techniques. This precise hardware-software integration solves the technical problem of secure memory sanitization in interpreted runtimes.

Core Cryptographic Engine (KEM / DEM Hybrid Implementation)

The core cryptographic engine is realized as an embedded module compiled from memory-safe systems programming instructions (specifically, Rust). To defend against both quantum and classical algorithmic analysis, the engine replaces solitary asymmetric calculations with a hybrid Key Encapsulation Mechanism (KEM) and Data Encapsulation Mechanism (DEM) pipeline.
When an encryption operation is initiated within the core module:

1. An operational Key Encapsulation Mechanism based on Module-Lattice-Based cryptography—specifically aligning with NIST ML-KEM / CRYSTALS-Kyber-1024 parameters—is invoked. The KEM layer accepts an asymmetric 1,568-byte public key (PK) and mathematically generates two artifacts: an encapsulated post-quantum ciphertext (CT_{kem}) of 1,568 bytes, and a high-entropy symmetric Shared Secret (SS) of 32 bytes (256 bits).
2. The engine immediately invokes an Operating System cryptographic Random Number Generator (OS-RNG) to extract a 12-byte (96-bit) pseudorandom Number Once (Nonce / IV).
3. The 32-byte Shared Secret (SS) is injected as the symmetric cryptographic key into an Authenticated Encryption with Associated Data (AEAD) cipher—specifically Advanced Encryption Standard in Galois/Counter Mode (AES-256-GCM).
4. The DEM layer transforms the arbitrary client plaintext data into an encrypted symmetric ciphertext (CT_{dem}) accompanied by an authenticated message integrity tag.
5. Prior to exiting the core native routine, intermediate stack allocations holding the 32-byte Shared Secret (SS) are actively zeroized and purged from RAM registers to prevent memory-dump extraction attacks.

Operational Mode 1: Ephemeral Vault Mode (Permanent Sealing Architecture)

A primary technical innovation of the present invention resides in its memory-enforced key lifecycle for immutable audit records (Vault Mode). Conventional cryptographic libraries require developers to externally generate, store, and manage encryption keys, leaving historical logs vulnerable if server storage is subsequently compromised or legally subverted.
In the disclosed **Vault Mode**:

1. When the client application invokes the `vault_encrypt` processing routine, the core engine creates an internal, temporary memory sandbox on the local call stack.

2. An ephemeral Kyber-1024 public key (PK_{eph}) and corresponding ephemeral private key (SK_{eph}, spanning 3,168 bytes) are generated inside this local sandbox.
3. The KEM/DEM encapsulation executes using PK_{eph}, encrypting the sensitive transaction payload (e.g., healthcare patient logs, banking transfers).
4. **Deterministic Memory Release:** Immediately upon successful derivation of the ciphertext string, the engine invokes a deterministic deallocation command on the memory address space occupied by SK_{eph}. The ephemeral private key is strictly barred from being serialized, exported, returned to the calling application, or written to non-volatile disk storage.
5. Consequently, once the function returns the formatted payload string prefixed with the identifier `QZ_VAULT_V1:`, the computational system itself enters a state of mathematical immutability. No entity—not the client software, not system administrators, and not the network ingestion server—can ever decapsulate or decrypt the record. The ciphertext acts as a verifiable, permanently sealed compliance voucher.

Operational Mode 2: Persistent Secure Mode (End-to-End Architecture)

For bidirectional enterprise communication requiring authorized data retrieval, the engine activates **Secure Mode**:

1. A client invokes `generate_keypair()`, prompting the engine to generate persistent ML-KEM cryptographic keys encoded into standard Base64 representation: a 1,568-byte public key (PK_{pers}) and a 3,168-byte private key (SK_{pers}).
2. The user client architecture safely retains SK_{pers} inside client-side environment memory or isolated hardware storage.
3. To communicate securely, the sender invokes `secure_encrypt(plaintext, recipient_PK)` producing a formatted payload prefixed with `QZ_SECURE_V1:`.
4. The encrypted payload is transmitted over standard networks. Because encryption executes 100% locally within client memory prior to transmission, network interceptors collecting data (HNDL attacks) acquire only quantum-resistant lattice ciphertexts.
5. Upon retrieval, the receiving client invokes `secure_decrypt(payload, recipient_SK)`. The core module locally extracts the KEM ciphertext, decapsulates the 32-byte Shared Secret (SS) using SK_{pers}, verifies the AES-GCM integrity tag against tampering, and returns the recovered plaintext directly into client volatile memory.

Cross-Runtime Bridge and Capability Negotiation Layer

To solve ecosystem fragmentation without duplicating core algorithms, the system introduces an intermediary **Runtime Binding Layer** that transforms native Rust instructions into cross-platform execution bridges:

- **WebAssembly (WASM) JavaScript Injection:** Using automated binding generators (`wasm-bindgen`), the Rust cryptographic core is transpiled into a highly optimized `.wasm` binary instructional file accompanied by TypeScript type definition files (`.d.ts`) and JavaScript linkage modules. Utilizing a **Zero-Config Injection** process, the WASM binary is natively bundled into frontend web application distributions and Node.js server architectures. To ensure execution stability across evolving software installations, the WASM wrapper incorporates dynamic runtime capability detection (e.g., an instructional check verifying whether extended dual-mode routines exist within the currently compiled WebAssembly runtime memory before invoking calls).
- **Native Python Extension (FFI Bridge):** Concurrently, the engine utilizes Foreign Function Interface architecture (`pyo3` / `maturin`) to link native Rust library symbols directly into CPython extension modules (`.so` on Linux/macOS, `.pyd` on Windows).
- **Interoperable Payload Schema:** Regardless of which runtime bridge executes the cryptography, all outputs adhere strictly to a standardized lexical syntax: $\text{Schema: } \langle\text{Mode_Prefix}\rangle : \langle\text{Base64_Kyber_CT}\rangle : \langle\text{Base64_AES_Nonce}\rangle : \langle\text{Base64_AES_CT}\rangle$
  This uniform technical structuring ensures that an encrypted string generated within a web browser via WebAssembly can be natively ingested, verified, or decrypted by a Python backend microservice or a compiled Rust server without translation overhead or schema incompatibility.

Untrusted Network Ingestion Server and Cryptographic Receipts

Prior art cloud security platforms require client applications to trust central server gateways with secret keys or raw plaintext data during ingestion and telemetry tracking. The present invention flips this paradigm via an **Untrusted Network Ingestion Architecture**:

1. **Zero Key Storage:** The backend API Gateway server is structured with zero decapsulation algorithms and zero key-management database tables. It exists solely as an authenticated ingestion boundary and metadata logger.
2. **Environment Isolation:** The gateway authenticates incoming HTTP REST ingest requests via isolated API key schemas designated by statutory prefix tokens (e.g., `qz_live_...` for production environments and `qz_test_...` for staging experimentation). These API keys govern rate limiting and enterprise quotas in a persistent document database (MongoDB), bearing zero mathematical linkage to the cryptographic encryption keys.
3. **Boundary Schema Verification:** When a client transmits a payload to the gateway ingestion route (`POST /api/v1/ingest`), the gateway executes strict lexical pattern verification on the payload string. If an incoming payload lacks an authentic QuantaCipher schema tag (`QZ_VAULT_V1:`, `QZ_SECURE_V1:`, or backwards-compatible `QZ_TRUE_PQC_KEM:`), the network boundary immediately rejects the packet with an HTTP protocol exception. Plaintext data is structurally blocked from gaining admission to server infrastructure.
4. **Turnkey Cryptographic Receipts:** Upon successful schema authentication and payload size counting (in bytes), the gateway server constructs and returns an immutable **Cryptographic Receipt**. This receipt represents a non-repudiable audit artifact containing:
   - A unique receipt identifier token (`qz_rcpt_<timestamp>_<random_entropy>`);
   - An exact verified server timestamp;
   - An algorithm verification tag confirms the cipher structure (e.g., "`Kyber-1024 + AES-256-GCM`");
   - An exact enumeration of bytes secured.
     This mechanism permits enterprise users to furnish regulators (e.g., under healthcare HIPAA audits or SOC2 compliance evaluations) with cryptographic proof that data was post-quantum encrypted at a specific point in time, without ever exposing the underlying sensitive data to the auditing server.

(Note: Claims must start on a separate page)

5. CLAIMS

I/We Claim:

1. A computer-implemented system for cross-runtime post-quantum cryptographic data isolation and verifiable network ingestion, the system comprising:
   - **(a) a local client cryptographic engine** executable inside an application memory address space of a client processing device, configured to perform hybrid key encapsulation and symmetric data encryption utilizing a post-quantum Key Encapsulation Mechanism (KEM) to generate a shared secret and a Data Encapsulation Mechanism (DEM) to encrypt plaintext data using said shared secret, wherein said engine operates across at least two distinct memory-enforced key lifecycle modes;
   - **(b) a cross-runtime compilation binding layer** configured to wrap said local client cryptographic engine into a plurality of targeted runtime executables, including a WebAssembly (WASM) module for JavaScript runtime environments and a native foreign function interface extension for Python runtime environments, wherein each runtime executable exposes uniform functional interfaces and generates an interoperable structured ciphertext schema; and
   - **(c) an untrusted network API gateway server** positioned across a communications network, configured to receive structured ciphertext schemas from said runtime executables, validate schema formatting at a network boundary without possessing decapsulation keys or accessing unencrypted plaintext, and return an immutable timestamped cryptographic audit receipt upon verification.
2. The system as claimed in claim 1, wherein the post-quantum Key Encapsulation Mechanism (KEM) comprises a Module-Lattice-Based Key Encapsulation algorithm conforming to ML-KEM / Kyber-1024 parameters, generating a 32-byte high-entropy shared secret and a 1,568-byte encapsulation ciphertext.

3. The system as claimed in claim 1, wherein the Data Encapsulation Mechanism (DEM) comprises an Advanced Encryption Standard cipher operating in Galois/Counter Mode (AES-256-GCM), utilizing the 32-byte shared secret as a symmetric encryption key and applying a dynamically generated 12-byte random nonce.
4. The system as claimed in claim 1, wherein a first mode of the local client cryptographic engine is an **Ephemeral Vault Mode**, wherein the engine is configured to:
    - generate an ephemeral asymmetric post-quantum keypair within an isolated function memory scope;
    - encrypt the incoming plaintext utilizing the ephemeral public key;
    - immediately execute a deterministic deallocation and zeroization of the ephemeral private key from system memory upon ciphertext generation; and
    - output an permanently sealed ciphertext payload tagged with a distinctive vault prefix token (`QZ_VAULT_V1:`), whereby decapsulation is irreversibly precluded across all computational entities.
5. The system as claimed in claim 1, wherein a second mode of the local client cryptographic engine is a **Persistent Secure Mode**, wherein the engine is configured to encrypt plaintext utilizing a persistent post-quantum public key provided by the calling application, outputting a ciphertext payload tagged with a secure prefix token (`QZ_SECURE_V1:`), wherein said payload is locally decapsulatable exclusively within the application memory address space of a client holding a corresponding persistent private key.
6. The system as claimed in claim 1, wherein the interoperable structured ciphertext schema generated by all targeted runtime executables conforms to a uniform string format comprising: an operational mode prefix tag, a delimiter, a base64-encoded representation of the post-quantum KEM ciphertext, a delimiter, a base64-encoded representation of the symmetric initialization nonce, and a delimiter followed by a base64-encoded representation of the symmetric encrypted data payload and integrity tag.
7. The system as claimed in claim 1, wherein the WebAssembly (WASM) module executed within JavaScript and Node.js environments comprises a runtime capability detection module configured to programmatically inspect the compiled WebAssembly memory table to verify the availability of dual-mode encryption functions prior to execution.
8. The system as claimed in claim 1, wherein the untrusted network API gateway server is completely devoid of post-quantum private key material and decapsulation logic, and enforces zero-trust data isolation by applying lexical boundary checks that automatically discard and terminate network connections for incoming transmissions failing to match authorized QuantaCipher structural prefixes (`QZ_VAULT_V1:`, `QZ_SECURE_V1:`, or `QZ_TRUE_PQC_KEM:`).
9. The system as claimed in claim 1, wherein the API gateway server authenticates ingestion requests using an environment-isolated API key framework comprising distinct prefix designations for production and test staging environments (`qz_live_` and `qz_test_`), wherein said API keys govern rate-limiting thresholds and billing usage counters in a connected document database without possessing any mathematical relation to cryptographic encryption key pairs.
10. The system as claimed in claim 1, wherein the immutable timestamped cryptographic audit receipt generated by the network server comprises: an authentic unique receipt identifier token (`qz_rcpt_`), an ISO-formatted verification timestamp, an identifier string defining the authenticated cryptographic algorithm combination, and an exact numerical count of ciphertext payload bytes ingested.
11. A computer-implemented method for cross-runtime post-quantum cryptographic key lifecycle orchestration and zero-trust network data ingestion, the method comprising the steps of:
    - **(i) initializing** a local client cryptographic engine inside a client application memory address space via an imported runtime binding module compiled from a foundational systems language into either a WebAssembly binary or a native language extension;
    - **(ii) selecting** an operational encryption mode from between a permanent sealing Vault Mode and an end-to-end Secure Mode;
    - **(iii) encapsulating** a post-quantum symmetric shared secret and encrypting local plaintext data using a hybrid KEM/DEM cryptographic scheme running entirely within local client memory, preventing plaintext departure from the local runtime;
    - **(iv) structuring** the resulting ciphertext into a cross-runtime compatible string payload bearing a mode-specific lexical prefix and base64-encoded cryptographic components;
    - **(v) transmitting** solely the structured ciphertext payload over a network to an untrusted API gateway server; and
    - **(vi) validating** the structured schema at the API gateway server without decrypting the payload, recording byte telemetry, and returning a verified cryptographic receipt back to the calling client application.
12. The method as claimed in claim 11, wherein selecting the permanent sealing Vault Mode automatically triggers a step of deterministically zeroizing and dropping an internally generated ephemeral post-quantum private key immediately following ciphertext encapsulation, thereby rendering the historical ciphertext record mathematically permanent and undecryptable.
13. The method as claimed in claim 11, wherein the step of validating at the untrusted API gateway server further comprises inspecting HTTP headers for environment-scoped API keys, verifying usage quotas against an external database, and rejecting raw unencrypted plaintext payloads prior to application server processing.
14. The method as claimed in claim 11, wherein the hybrid KEM/DEM scheme employs Module-Lattice-Based Key Encapsulation (ML-KEM / Kyber-1024) to establish a 256-bit secret key that directly parametrizes an AES-256-GCM symmetric encryption algorithm utilizing a 96-bit OS-generated random initialization nonce.

(Note: Date and Signature must be at the end of the claims page)

6. DATE AND SIGNATURE

Dated this _____ day of _____, 20___.

Signature: _____
Name: QuantaLabs Private Limited / Authorized Patent Agent
Registration No: IN/PA/_____

(Note: Abstract must start on a separate page)

7. ABSTRACT OF THE INVENTION

A zero-trust post-quantum cryptographic system and method operable across heterogeneous computing runtimes without server-side key exposure. A local client cryptographic engine implements a hybrid KEM/DEM scheme (NIST ML-KEM Kyber-1024 + AES-256-GCM) compiled via runtime bridges into WebAssembly (WASM) for JavaScript/browser environments and native FFI modules for Python. The engine operates in an ephemeral Vault Mode, which deterministically drops temporary private keys upon encryption to produce non-repudiable sealed audit records, or a persistent Secure Mode for local end-to-end decapsulation. All cryptography executes within local client memory; unencrypted plaintext never leaves the runtime. An untrusted network API gateway validates structured ciphertext schemas (`QZ_VAULT_V1:`, `QZ_SECURE_V1:`) via lexical prefix inspections without holding decapsulation keys, meters data throughput, and issues immutable timestamped cryptographic audit receipts to verify compliance.